The Provenance Forest

Poker's generalisation step without enumerating refutations

Oruži Zarathustra Goertzel

7 November 2026

Abstract

Poker generalises an example by collecting, for every refutation of the example within the clause limit, the set of metasubstitutions the refutation uses. Vanilla enumerates these refutations one at a time, and their number grows exponentially with the clause limit. The provenance forest computes the same sets by dynamic programming: it solves each first-order goal once for each exact size, and builds the sets of a refutation from those of its parts. It reads Vanilla's own problem and runs behind Poker's `generalise/3`. On Poker's experiments and on 80 random grammar-learning problems its output is identical to Poker's in every case compared, for example the same 9,941 sub-hypotheses of a^3b^3 at clause limit 7 in 0.69 s where Poker takes 225 s.

1 The problem

Poker is the self-supervised grammar learner of the Vanilla framework for meta-interpretive learning [3], with Louise [4]. Its generalisation step takes an example e, background knowledge B, a set of metarules, a signature (the target predicate and the invented predicates) and a clause limit K. Each refutation of e resolves goals either against B or against an instance of a metarule; each such instance is a *metasubstitution* σ, the metarule with its second-order variables bound. The *support* of a refutation is the set of metasubstitutions it uses. Poker's sub-hypotheses are the supports of the refutations of e with at most K steps.

Vanilla finds them by SLD-resolution with the metasubstitutions found so far as an argument of every call. Distinct refutations share most of their sub-refutations, but an untabled prover derives each shared part again in every refutation that contains it. Tabling the meta-interpreter does not recover the sharing: two calls on the same goal with different stores are different calls. On a^4b^4 at clause limit 7 Poker spends 5.13 billion inferences to produce 18 sets.

2 The decomposition

A refutation is a tree. Its root is a metasubstitution σ whose head matches the goal; its children are refutations of the body literals of σ; its leaves are background facts. Size and support are both compositional:

$$\mathrm{size}(t) = 1 + \sum_i \mathrm{size}(t_i), \qquad \mathrm{supp}(t) = \{\sigma\} \cup \bigcup_i \mathrm{supp}(t_i),$$

with size 0 and support $\emptyset$ at a background leaf. The size counts steps with multiplicity, while the support is a set, as in Poker's store. Neither depends on the rest of the tree, so the family of

supports of a goal's refutations of size exactly n,

$$F_n(g) = \{\, \mathrm{supp}(t) : t \text{ a refutation of } g,\ \mathrm{size}(t) = n \,\},$$

satisfies

$$F_0(g) = \{\emptyset\} \text{ if } g \in B, \text{ and } \emptyset \text{ otherwise,}$$
$$F_{n+1}(g) = \bigcup_{\sigma : g \leftarrow g_1,\ldots,g_m} \ \bigcup_{n_1+\cdots+n_m=n} \{\, \{\sigma\} \cup S_1 \cup \cdots \cup S_m : S_i \in F_{n_i}(g_i) \,\},$$

and the generalisation of e at clause limit K is $\bigcup_{n=1}^{K} F_n(e)$. Each $F_n(g)$ is computed once and shared by every refutation that contains g at size n.

Theorem 1 (the recurrence characterises the supports)**.** *For a ground derivation forest (goals, background goals, and edges $g \leftarrow \sigma, [g_1, \ldots, g_m]$), the family F of supports of derivation trees satisfies the recurrences above, and every family satisfying them equals F.*

Proof sketch. That F satisfies the recurrences is the decomposition of a tree at its root. For uniqueness, let G satisfy them and show $G_n(g) = F_n(g)$ for all g by strong induction on n: at $n = 0$ both are given by B; at $n + 1$ both sides apply the same recurrence to families of sizes $n_i \leq n$, which agree by induction. □

Lemma 2 (pruning loses nothing)**.** *In the recurrence, a splitting $n_1 + \cdots + n_m = n$ contributes a set only if each body literal g_i has a derivation of size n_i. Hence restricting the splittings to those in which every literal is known to be derivable at its share keeps every set of $F_{n+1}(g)$.*

Both statements are machine-checked in Lean 4 with Mathlib, using no axioms beyond Lean's standard ones, together with positive and negative examples.

3 From ground forests to MIL goals

In meta-interpretive learning the forest is not given in advance. A goal is an encapsulated literal $P(a_1, \ldots, a_k)$ with some arguments bound; the body of Chain, $Q(X, Z), R(Z, Y)$, introduces an existential Z that is known only once $Q(x, Z)$ is solved. The forest therefore works in two passes over memo tables keyed by goal (up to variable renaming) and size.

Pass 1: instances. *reach*(g, n) is the set of instances of the goal g that have a refutation of size exactly n. A body is solved left to right: the size $n-1$ is split among the literals, and the instances of each literal at its share bind the variables the next literals share with it. A literal of size 0 is answered by B; a literal of positive size is a target or invented predicate, answered by *reach* itself. A literal is memoised as its call pattern: when an argument before the last is bound, the last is left open, as the output of a grammar rule or a function, so that every call on the same input shares one entry. Solving a more general goal and unifying its answers with the literal gives exactly the literal's answers, so the pattern affects only the cost.

Pass 2: supports. *fam*(i, n) is $F_n(i)$ for an instance i, built by the recurrence, but only over the splittings pass 1 found feasible. By the pruning lemma this loses nothing.

Invented predicates. Each memo key also carries which invented predicates are available: the i-th invented predicate may head a clause only once the $(i-1)$-th is available. This names inventions canonically, so that supports differing only by a renaming of invented predicates are not produced twice. A goal of the ground forest is then a goal together with this state, and the theorem applies unchanged.

Metarule constraints. Poker's constraints, such as $P \neq Q$ in Chain, are tests on a single metasubstitution. The forest calls Vanilla's own check on each metasubstitution when it is built and again as its body literals bind its symbols; the constraints compare symbols with `==`, so that a symbol not yet bound passes. A constraint on one metasubstitution removes edges from the forest and does not interact with the decomposition.

4 Behind Poker's API

The forest reads a Vanilla MIL problem as Poker does: the encapsulated metarules, the background knowledge Vanilla writes to the `user` module, the configuration's metarule constraints, and Poker's signature of the target and its invented predicates. `forest_generalise/3` answers as Poker's `generalise/3`: it computes each positive example's sub-hypotheses with the forest and passes them through Poker's own constraint check, sorting, unfolding and renaming of invented predicates, so the two outputs can be compared as Prolog terms. `use_forest/0` wraps `generalise/3`, so Poker's `learn/1` runs on the forest unchanged; it falls back to Vanilla wherever the forest does not apply.

The forest applies where Vanilla resolves goals against background knowledge and metarules only (`fetch_clauses` without `hypothesis`), keeps every proof, and generalises examples one at a time without threads. Some of Poker's experiment settings are written that way: $a^n b^n$, parity, the *even* setting of the weak Chomsky–Greibach script, the *palindrome_bin_ng* setting of the binary script, and five settings of the Lindenmayer script. Most of the others, hello world and almost all of the Chomsky–Greibach grammars among them, use Vanilla's default, which also resolves goals against hypothesis clauses already built. There the clause limit bounds the distinct metasubstitutions of a refutation, not its steps, and size is no longer additive; the sub-hypotheses without reuse are a strict subset of Poker's (hello world: 16 of 18; the Chomsky–Greibach parentheses: 64 of 256).

With reuse. The forest then computes the least fixpoint of the same recurrence without sizes: a goal's supports are the sets $\{\sigma\} \cup S_1 \cup \cdots \cup S_m$ of at most K metasubstitutions, over clauses $\sigma : g \leftarrow g_1, \ldots, g_m$ and supports S_i of the g_i, and of background facts the empty set. SWI-Prolog's tabling computes it, cycles included, each literal tabled as its call pattern; it is finite because supports are bounded sets over finitely many metasubstitutions. The invention state is threaded in Vanilla's depth-first order as before: a clause headed by the i-th invented predicate is introduced only once the $(i-1)$-th is available, and a clause met again passes, its head already available. A check on the set alone (that the inventions used are inv_1 to inv_j) is too weak: it admits 342 sub-hypotheses for the parentheses where Poker has 256.

Theorem 3 (the bounded fixpoint characterises the supports with reuse)**.** *For a ground derivation forest and a bound K, the least family closed under the recurrence without sizes, every support and every partial union of a body kept within K labels, is exactly the set of supports of derivation trees with at most K labels; a label may occur any number of times in a tree.*

The proof is by induction both ways: a subtree's support is part of its tree's, so the bound prunes no tree whose support stays within it. It is machine-checked in Lean 4 with Mathlib, with

Lean's standard axioms only, beside the theorem of Section 2; that the depth-first invention state makes the forest's items correspond to Vanilla's refutations is checked against Poker, not proved.

5 Why-provenance

$F_n(g)$ is the why-provenance of g (Buneman, Khanna and Tan, 2001), restricted to derivations of size n, with metasubstitutions in the place of database tuples. In the semiring view of provenance (Green, Karvounarakis and Tannen, 2007) its values are sets of sets, added by union over alternative derivations and multiplied by pairwise union over the literals of a body. Grading by size keeps the family finite for recursive targets, and the theorem above is the statement that the graded fixpoint is the provenance of the derivation trees.

6 Cost

The memo tables hold one entry per goal, size and invention state. Pass 1 costs, per entry, one step for each metarule, choice of symbol, splitting of the size and intermediate endpoint. Pass 2 builds only sets that are parts of sub-hypotheses: every set it builds is contained in one of the output's sets. Poker's cost grows with the number of refutations instead, and of the partial ones that fail, a number that grows exponentially with K even when the number of distinct supports does not: at a^4b^4, limit 7, there are 18 supports.

Table 1 reports Poker's `generalise/3` and `forest_generalise/3` on the same problems, in SWI-Prolog 10.1.9. Their outputs, Poker's sub-hypotheses after its own sorting and renaming, are identical as Prolog terms in every row; the forest's times include that post-processing. At limits too small to fit any refutation neither has an answer: Poker's `generalise/3` fails, and so does the forest's, through the same post-processing (a^3b^3 at $K = 4$: 97.7 ms against 1.1 ms).

Problem	Sub-hypotheses	Poker		Forest	
		time	inferences	time	inferences
a^1b^1, $K = 1$	1	1.0 ms	6,164	0.1 ms	1,527
a^2b^2, $K = 3$	4	9.5 ms	199,910	0.9 ms	13,945
a^2b^2, $K = 5$	1,449	1.72 s	35,884,088	82 ms	1,931,700
a^3b^3, $K = 5$	10	1.21 s	29,254,138	3.3 ms	58,700
a^3b^3, $K = 7$	9,941	226 s	5,348,394,311	694 ms	17,197,057
a^4b^4, $K = 7$	18	210 s	5,134,352,920	9.9 ms	164,296
parity, $K = 5$	362	570 ms	4,971,836	68 ms	949,044

Table 1: Poker against the forest on Poker's a^nb^n experiment at several sizes and on its parity experiment (16 examples), single runs; the inference counts are exact.

With hypothesis reuse, Vanilla's default, the two outputs are identical on Poker's hello world and on four Chomsky–Greibach settings, with Poker's own unfolding of invented predicates applied to both:

On 80 random grammar-learning problems (terminals, metarules among Identity, Chain and Tri-chain, constraints, one example string of length 0 to 5, clause limits 1 to 4, 0 to 2 invented predicates) the two outputs are identical in all 80: 50 have sub-hypotheses, the largest 19,073; Poker takes 105 s in all and the forest 1.7 s. Poker's `learn/1` on parity learns the same five-clause grammar of even parity, with an invented predicate, on the forest as on Vanilla, in 173 ms against

Problem (reuse)	Sub-hypotheses	Poker	Forest
hello world, $K = 2$	18	4.4 ms	0.8 ms
C–GNF $a^n b^n$, $K = 3$	18	56 ms	35 ms
C–GNF $a^n b^m$, $K = 4$	70	102 ms	11 ms
C–GNF even parity, $K = 5$	508	3.2 s	0.14 s
C–GNF parentheses, $K = 4$	203	1.60 s	1.51 s

Table 2: Poker against the forest with hypothesis reuse (single runs).

629 ms. Without Poker's post-processing the forest computes the supports of $a^5 b^5$ at $K = 9$ in 16 ms and of $a^7 b^7$ at $K = 13$ in 67 ms.

Table 3 goes beyond Poker's own experiments, to task files written for Poker's setting from other benchmarks: the regular and context-free tasks of Delétang et al.'s *Neural Networks and the Chomsky Hierarchy* [5], with and without Chomsky–Greibach normal-form metarules; the kinship, robot and points-to tasks of Popper's suite [6]; list membership from Hopper's suite [7]; Louise's even and odd [3]; and a probe of the Inverse metarule. Each runs with its own clause limit and without reuse. The outputs are identical on the 13 tasks Poker finishes within 200 s. On the other three the forest alone was run.

Task	Sub-hypotheses	Poker	Forest
reverse string	18,959	92.1 s	1.50 s
parity check	21,740	32.6 s	1.57 s
even pairs, C–GNF	783	4.87 s	103 ms
reverse string, C–GNF	183	649 ms	44 ms
cycle navigation, C–GNF	266	363 ms	64 ms
parity check, C–GNF	227	191 ms	37 ms
robots, linear (Popper)	35	8.89 s	5.4 ms
even and odd (Louise)	45	15.3 ms	2.4 ms
grandparent with invention (Popper)	55	11.2 ms	4.2 ms
points-to, Andersen (Popper)	22	9.5 ms	2.5 ms
list membership (Hopper)	2	3.4 ms	5.2 ms
ancestor (Popper)	3	1.4 ms	0.5 ms
Inverse probe	1	0.9 ms	0.1 ms
cycle navigation	1,581,602*	not in 200 s	8.6 s*
even pairs	1,125,904*	not in 200 s	3.7 s*
robots, recursion (Popper)	3*	not in 200 s	98 ms*

Table 3: Poker against the forest on task files from other benchmarks (single runs, the forest's time including Poker's post-processing). *Distinct supports and the time to compute them, before post-processing.

The forest is slower than Poker only on list membership, where Poker's search is shallow and the forest's memo tables cost more than they save.

7 Scope

- The decomposition assumes that a refutation resolves goals against background knowledge and metarules only, not against clauses of the hypothesis being built, so that every step instantiates a metarule and the clause limit bounds the steps. This is the setting of the $a^n b^n$ experiment: there Poker finds no refutation of $a^3 b^3$ at limit 4, although a grammar of three clauses derives it in five steps. With reuse the forest computes a least fixpoint instead (Section 4), whose ground law is proved; that the invention state threaded in depth-first order matches Vanilla's is checked against Poker, not proved.
- The forest replaces the generalisation step only. Poker's other stages are unchanged, and they still receive every sub-hypothesis as a list; they could work on the forest itself, which represents the same family compactly.

8 Running it

The forest is in `lib/forest`; tasks and scripts are in `data/forest_examples`. With Poker selected as the learner in `configuration.pl`, from Vanilla's root,

```
swipl data/forest_examples/compare_with_poker.pl TARGET EXPERIMENT [K]
```

compares the two `generalise/3` on an experiment file, for example with TARGET `s/2` and EXPERIMENT `data/poker_examples/anbn_poker.pl`;

```
swipl data/forest_examples/poker_comparison.pl [N [K]]
```

does so on $a^N b^N$ at any size; and

```
data/forest_examples/random_problems.sh
```

runs all 80 random problems. In a Prolog session, `use_forest` makes Poker's `learn/1` use the forest. The forest was first written in MeTTa, where each memo table is a space of shared immutable terms indexed by their arguments; the files here are its port to SWI-Prolog, with the memo tables as asserted facts keyed by a variant hash or as tabled predicates.

References

[1] P. Buneman, S. Khanna and W.-C. Tan. Why and where: a characterization of data provenance. ICDT 2001.

[2] T. J. Green, G. Karvounarakis and V. Tannen. Provenance semirings. PODS 2007.

[3] S. Patsantzis. Vanilla: an inductive logic programming framework, with the learners Louise and Poker. `https://github.com/stassa/vanilla`.

[4] S. Patsantzis and S. H. Muggleton. Top program construction and reduction for polynomial time meta-interpretive learning. Machine Learning 110, 2021.

[5] G. Delétang, A. Ruoss, J. Grau-Moya, T. Genewein, L. K. Wenliang, E. Catt, C. Cundy, M. Hutter, S. Legg, J. Veness and P. A. Ortega. Neural networks and the Chomsky hierarchy. ICLR 2023.

[6] A. Cropper and R. Morel. Learning programs by learning from failures. Machine Learning 110, 2021.

[7] S. J. Purgał, D. M. Cerna and C. Kaliszyk. Learning higher-order logic programs from failures. IJCAI 2022.